

State Management Blueprint

Full Stack Development Internship Report

Submitted to:

Prosensia

Submitted by:

Tayyaba Shakoor

Full Stack Development Intern

04 June 2026

1. Introduction to State Management

State management is how applications handle and share data across components. In React, choosing the wrong state management strategy can cause performance issues, unnecessary re-renders, and complex debugging.

There are three main types of state:

State Type	Examples	Best For
Local State	useState, useReducer	Component-specific data
Server State	React Query, SWR, Apollo	Data from APIs
Global Client State	Zustand, Redux, Context API	Shared UI state across components

2. Local State (useState)

What It Is

useState is React's built-in hook for managing state inside a single component.

Syntax Example

```
const [count, setCount] = useState(0);
const [user, setUser] = useState({ name: "", age: 0 });
```

When to Use

- Toggle switches (modal open/close)
- Form input values
- Loading states
- UI flags (dropdown open)

Performance Boundaries

Aspect	Boundary
Re-render scope	Only the component and its children
Memory	Cleaned up when component unmounts
Re-render trigger	Every setState call triggers re-render

Good Example

```
function ToggleButton() {
  const [isOn, setIsOn] = useState(false);
  return <button onClick={() => setIsOn(!isOn)}>{isOn ? "ON" : "OFF"}</button>;
}
```

BAD Example — Causes Re-render Loops

```
function BadComponent() {
  const [count, setCount] = useState(0);

  // This causes infinite loop
  setCount(count + 1);

  return <div>{count}</div>;
}
```

Why it loops: setState inside render triggers another render, which triggers setState again.

3. Server State (React Query)

What It Is

React Query (now called TanStack Query) is a library for fetching, caching, and synchronizing server data.

Why Not useState for API Data?

useState alone cannot handle:

- Caching
- Background refetching
- Loading and error states
- Pagination
- Optimistic updates

React Query Syntax

```
import { useQuery } from '@tanstack/react-query';

const { data, isLoading, error } = useQuery({
  queryKey: ['users'],
  queryFn: () => fetch('/api/users').then(res => res.json())
});
```

Performance Benefits

Feature	Benefit
Automatic caching	Reduces network requests
Background refetching	Data stays fresh
Stale-while-revalidate	Shows cached data while fetching new
Garbage collection	Cleans up unused data

Performance Boundaries

Aspect	Boundary
Re-render scope	Only components using the query
Network requests	Deduped automatically
Memory	Managed by React Query cache (configurable TTL)

When to Use React Query

- Fetching data from APIs
- Paginated lists
- Infinite scrolling
- Real-time updates
- Form submissions with optimistic updates

4. Global Client State (Zustand / Redux)

4.1 Redux

Redux is a predictable state container with a strict unidirectional data flow.

Redux Flow:

Action → Reducer → Store → View

Redux Syntax Example

```
// Action
const increment = { type: 'INCREMENT' };

// Reducer
const counterReducer = (state = 0, action) => {
  switch (action.type) {
    case 'INCREMENT': return state + 1;
    default: return state;
  }
};

// Store
const store = createStore(counterReducer);
```

- Pros: Predictable, great devtools, large ecosystem
- Cons: Boilerplate code, learning curve

4.2 Zustand

Zustand is a minimal global state management library.

Zustand Syntax Example

```
import { create } from 'zustand';

const useStore = create((set) => ({
  count: 0,
  increment: () => set((state) => ({ count: state.count + 1 })),
}));

function Counter() {
  const { count, increment } = useStore();
  return <button onClick={increment}>{count}</button>;
}
```

- Pros: Simple, less boilerplate, fast
- Cons: Smaller ecosystem than Redux

Performance Boundaries Comparison

Aspect	Redux	Zustand
Re-render scope	Components using useSelector	Components using the store hook
Bundle size	~12KB	~3KB
Learning curve	Steep	Gentle
Devtools	Excellent	Good

When to Use Global State

- User authentication status
- Theme (dark/light mode)
- Shopping cart
- Notifications/snackbars
- UI state shared across distant components

5. Performance Boundaries Comparison

5.1 When Each Type Re-renders

State Type	Triggers Re-render In
useState	Only that component + children
React Query	Only components using the query
Zustand	Only components using specific state slices
Redux	Only components using useSelector

5.2 Memory and Performance Impact

State Type	Memory Footprint	Performance
useState	Very low	Excellent
React Query	Medium (cached data)	Very good
Zustand	Low	Excellent

State Type	Memory Footprint	Performance
Redux	Medium	Good

5.3 Decision Framework

Ask these questions to choose the right state management:

Question	Answer → Use
Is this state only needed in one component?	Yes → useState
Is this state coming from an API?	Yes → React Query
Is this state needed across many components?	Small app → Zustand Large app → Redux

6. Preventing Catastrophic UI Layout Re-render Loops

6.1 What is a Re-render Loop?

A re-render loop happens when a state update causes another state update, which causes another render, creating an infinite cycle.

6.2 Common Causes

Cause	Example
setState inside useEffect without dependency	useEffect(() => setCount(count+1))
Object/array re-creation in props	<Child data={} />
Missing dependency array	useEffect(() => fetchData())
State update during render	setCount(count + 1) inside component body

6.3 BAD Code Example — Infinite Loop

```
function BadComponent() {  
  const [count, setCount] = useState(0);
```

```
// Infinite loop
useEffect(() => {
  setCount(count + 1);
}); // Missing dependency array

return <div>{count}</div>;
}
```

What happens:

- Component mounts
- useEffect runs after render
- setCount updates state
- Component re-renders
- useEffect runs again — loop continues forever

6.4 GOOD Code Example

```
function GoodComponent() {
  const [count, setCount] = useState(0);

  // Runs once
  useEffect(() => {
    setCount(prev => prev + 1);
  }, []); // Empty dependency array

  return <div>{count}</div>;
}
```

6.5 Loop Prevention Patterns

Pattern 1: Use Functional Updates

```
// Bad - depends on current state value
setCount(count + 1);

// Good - uses previous state
setCount(prev => prev + 1);
```

Pattern 2: Memoize Objects and Arrays

```
// Bad - new object every render
<Child data={{ name: "John" }} />

// Good - same object reference
const data = useMemo(() => ({ name: "John" }), []);
```



```
<Child data={data} />
```

Pattern 3: Memoize Functions

```
// Bad - new function every render
<Child onClick={() => console.log("clicked")} />

// Good - same function reference
const handleClick = useCallback(() => console.log("clicked"), []);
<Child onClick={handleClick} />
```

Pattern 4: Use React.memo for Expensive Components

```
export default React.memo(ExpensiveComponent);
```

7. Architecture Blueprint

Recommended State Management Architecture for a Medium App:

Server State (React Query) Handles all API data fetching, caching, and synchronization. Keeps network layer clean and efficient.	Client State (Zustand) Handles shared UI state such as authentication, theme, cart, and notifications.
UI Components (useState for local state only) Component-level state kept isolated. No external state unless strictly necessary.	

Data Layer Rules

Rule	Explanation
Rule 1	API data → React Query only
Rule 2	Shared UI state → Zustand/Redux
Rule 3	Component-specific state → useState
Rule 4	Never put API data in Redux/Zustand
Rule 5	Never put UI toggle state in React Query

Recommended Folder Structure

```
src/
├── store/
│   ├── authStore.ts      (Zustand - user auth)
│   ├── themeStore.ts     (Zustand - dark/light mode)
│   └── cartStore.ts      (Zustand - shopping cart)
├── hooks/
│   ├── useUsers.ts       (React Query - fetch users)
│   ├── useProducts.ts    (React Query - fetch products)
│   └── useOrders.ts      (React Query - fetch orders)
├── components/
│   └── ...
└── app/
    └── ...
```

8. Conclusion

Key Takeaways

- `useState` is for local, component-specific state. It is fast and simple but causes re-renders only in its component tree.
- `React Query` is for server state. It provides caching, background updates, and prevents unnecessary network requests.
- `Zustand/Redux` are for global client state shared across many components. `Zustand` is simpler for small apps; `Redux` is better for large enterprise apps.
- Performance boundaries depend on where state lives. Local state re-renders one component. Global state can re-render many if not optimized.
- Prevent re-render loops by using proper dependency arrays, functional updates, memoization, and never calling `setState` during render.

Final Blueprint for Your Prosensia Project

State Type	Used For	Implementation
<code>useState</code>	Modal open/close, form inputs	Already using
<code>React Query</code>	Future API data (users, projects)	Not yet, can add
<code>Zustand</code>	Theme (dark/light mode)	Not yet, can add

One-Line Summary

"Local state with useState for component data, React Query for server data caching, and Zustand for shared client state — never mix them, and always prevent re-render loops with proper dependency arrays and memoization."

Report Prepared By: Tayyaba Shakoor
Date: 04 June 2026 | Internship: Prosensia — Full Stack Development